COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 8, 2026
NAME:	Cardenas, Lea Marie N.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7 **SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 7-1. Array of Values for Sort Algorithm Testing

Screenshots	main.cpp > ...
	1 #include <iostream>
	2 #include <time.h>
	3 #include <cstdlib>
	4 #include <string>
	5 #include "sorting.h"
	6
	7 int main(){
	8 system("chcp 65001 > nul");
	9
	10 // RANDOM ARRAY
	11 const int size = 100;
	12 int randomArray[size];
	13
	14 srand(time(0));
	15
	16 for (int i = 0; i < size; ++i) {
	17 randomArray[i] = rand() % 100 + 1;
	18 }
	19
	20 std::cout << "Unsorted Random Array: \n";
	21 displayArray(randomArray, size);
	
	PS C:\Users\carde\OneDrive\Documents\7.1_Cardenas_DSA> g++ main.cpp -o main
	PS C:\Users\carde\OneDrive\Documents\7.1_Cardenas_DSA> .\main
	Unsorted Random Array:
	11 84 17 100 49 18 98 20 51 84 100 29 55 69 88 24 23 61 86 35 48 57 79 37 69 39 100 59 99 40 57 49 63 39 44
	45 51 40 96 21 29 97 24 43 38 88 64 14 85 65 46 58 22 33 100 21 14 83 21 68 8 57 77 88 41 7 35 69 32 12 98
	34 95 9 69 58 6 10 90 2 6 25 60 29 32 34 57 100 87 13 20 50 38 82 100 47 6 62 16 51

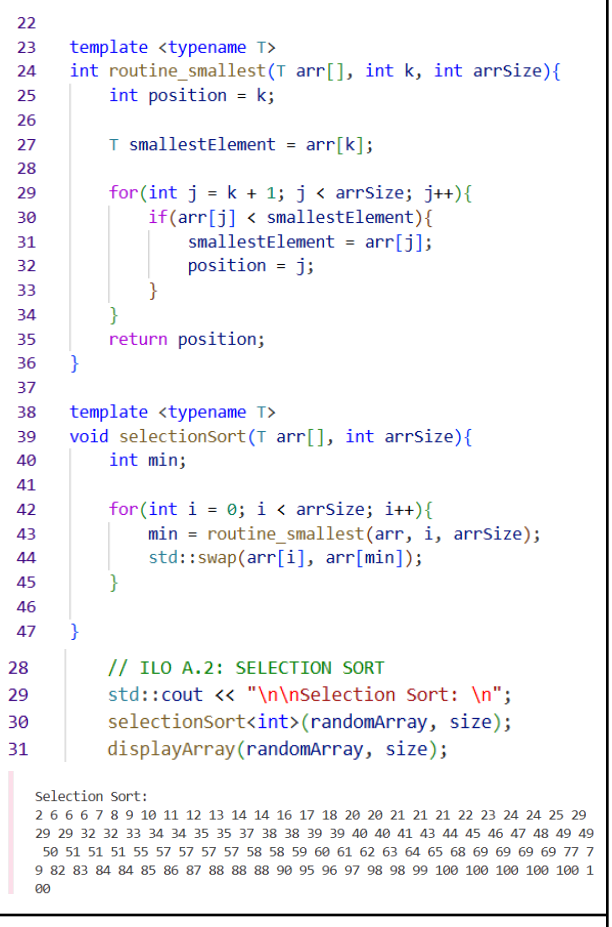
Observations	This random array contains numbers that are arranged in no particular order. In these values it is generated from 1 to 100, meaning the

	numbers may be repeated and some values may appear before or after larger numbers. The purpose of this sorting is to arrange these in a more organized way.

Table 7-2. Bubble Sort Technique

Screenshot	
Observation	This bubble sort means comparing numbers that are next to each other. Like when the first number is greater than the next number then the two numbers values will be swapped. As the process continues the larger number values gradually move towards the end of the array like in my screenshots. Then after the several passes the entire array becomes arranged from the smallest value to the largest value, like in my screenshots.

Table 7-3. Selection Sort Algorithm

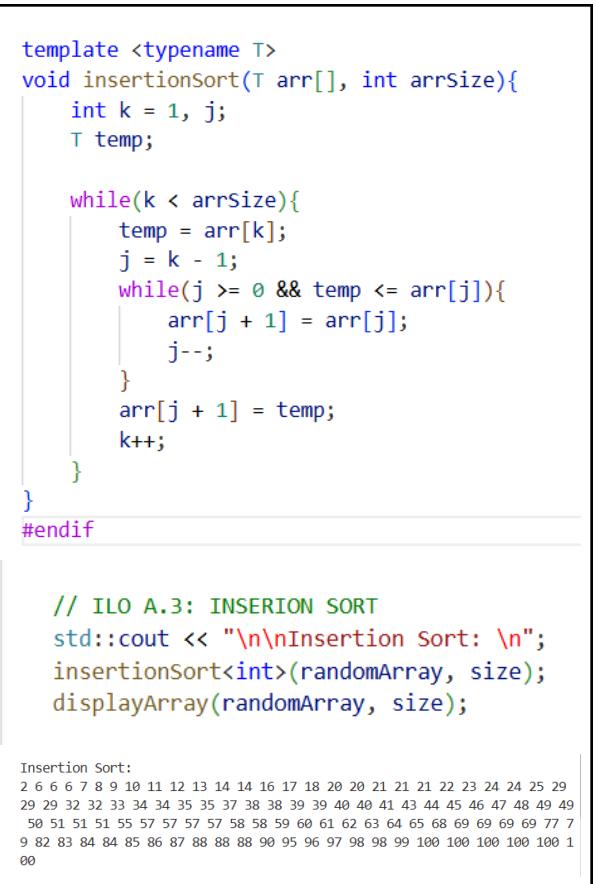
Screenshots	
Observations	This selection sorts means by looking for the smallest value unsorted portion of the array. Once the smallest value is found then the placed value is in the correct position like in the screenshots.then the sorted portion grows from the left to right until all the numbers are arranged in ascending order.

```
22
23  template <typename T>
24  int routine_smallest(T arr[], int k, int arrSize){
25      int position = k;
26
27      T smallestElement = arr[k];
28
29      for(int j = k + 1; j < arrSize; j++){
30          if(arr[j] < smallestElement){
31              smallestElement = arr[j];
32              position = j;
33          }
34      }
35      return position;
36  }
37
38  template <typename T>
39  void selectionSort(T arr[], int arrSize){
40      int min;
41
42      for(int i = 0; i < arrSize; i++){
43          min = routine_smallest(arr, i, arrSize);
44          std::swap(arr[i], arr[min]);
45      }
46
47  }
```

```
28      // ILO A.2: SELECTION SORT
29      std::cout << "\n\nSelection Sort: \n";
30      selectionSort<int>(randomArray, size);
31      displayArray(randomArray, size);
```

```
Selection Sort:
2 6 6 6 7 8 9 10 11 12 13 14 14 16 17 18 20 20 21 21 21 22 23 24 24 25 29
29 29 32 32 33 34 34 35 35 37 38 38 39 39 40 40 41 43 44 45 46 47 48 49 49
 50 51 51 51 55 57 57 57 57 58 58 59 60 61 62 63 64 65 68 69 69 69 69 77 7
9 82 83 84 84 85 86 87 88 88 88 90 95 96 97 98 98 99 100 100 100 100 100 1
00
```

Table 7-4. Insertion Sort Algorithm

Screenshots	
Observation	The array becomes more sorted gradually because each value is inserted into the correct position. So the portion sorted increases after every step until all the values are arranged from smallest to largest.

```
template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
#endif
```

```
// ILO A.3: INSERION SORT
std::cout << "\n\nInsertion Sort: \n";
insertionSort<int>(randomArray, size);
displayArray(randomArray, size);
```

```
Insertion Sort:
2 6 6 6 7 8 9 10 11 12 13 14 14 16 17 18 20 20 21 21 21 22 23 24 24 25 29
29 29 32 32 33 34 34 35 35 37 38 38 39 39 40 40 41 43 44 45 46 47 48 49 49
 50 51 51 51 55 57 57 57 57 58 58 59 60 61 62 63 64 65 68 69 69 69 69 77 7
9 82 83 84 84 85 86 87 88 88 88 90 95 96 97 98 98 99 100 100 100 100 100 1
00
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0...100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

SCREENSHOT OF ALGORITHM CODE:

```
37
38      // SUPPLEMENTARY ACTIVITY
39      const int voteSize = 100;
40      int votes[voteSize];
41
42      // Candidate names
43      std::string candidates[5] = {
44          "Bo Dalton Capistrano",
45          "Cornelius Raymon Agustín",
46          "Deja Jayla Bañaga",
47          "Lalla Brielle Yabut",
48          "Franklin Relano Castro"
49      };
50
51      // Generate random votes from 1 to 5
52      srand(12345);
53
54      for (int i = 0; i < voteSize; i++) {
55          votes[i] = rand() % 5 + 1;
56      }
57
58      // Display unsorted votes
59      std::cout << "\n\nUnsorted Votes:\n";
60      displayArray(votes, voteSize);
61      std::cout << "\n\n";
62
63      // Sort votes using insertion sort
64      insertionSort(votes, voteSize);
65
66      // Display sorted votes
67      std::cout << "Sorted Votes:\n";
68      displayArray(votes, voteSize);
```

```
    // Count votes for each candidate
    int voteCount[5] = {0};

    for (int i = 0; i < voteSize; i++) {
        voteCount[votes[i] - 1]++;
    }

    // Display election results
    std::cout << "===== ELECTION RESULTS =====\n";

    int winner = 0;

    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << i + 1 << ": "
                  << candidates[i] << " - "
                  << voteCount[i] << " votes\n";

        if (voteCount[i] > voteCount[winner]) {
            winner = i;
        }
    }

    // Display winner
    std::cout << "\nWINNER:\n";
    std::cout << candidates[winner]
              << " with " << voteCount[winner]
              << " votes.\n";

    return 0;
}
```

Output Console Showing Sorted Array

```
Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

Manual Count

```
===== ELECTION RESULTS =====
Candidate 1: Bo Dalton Capistrano - 24 votes
Candidate 2: Cornelius Raymon Agustín - 15 votes
Candidate 3: Deja Jayla Bañaga - 22 votes
Candidate 4: Lalla Brielle Yabut - 18 votes
Candidate 5: Franklin Relano Castro - 21 votes
```

Count Result of Algorithm

```
WINNER:
Bo Dalton Capistrano with 24 votes.
PS C:\Users\carde\OneDrive\Documents\7.1_Cardenas_DSA>
```

Question: Was your developed vote counting algorithm effective? Why or why not?

- Yes, the vote counting algorithm was effective because it successfully sorted the votes of each candidate and displayed the election results. The results were accurate based on the result code I did by visual code.

Conclusion:
This activity helped me to understand how sorting works by coding it all in visual code.I also learned how sorting can help in counting and organizing votes and showing the election exactly results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **I used chatgpt for my debugging and I used grammarly to check my sentences grammar**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT only as a guide for debugging and improving my code, the problems were done by me and the pdf guidelines. Grammarly was used only to check my grammar and clarity of my written explanations. AI tools did not replace my original works.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.